PROJECT REPORT: CAMPUS LOST AND FOUND SYSTEM (FINDIT)

Course: Database Management Systems Project

Authors:

Hasan Mert Gür 23291274

Mustafa Eren Emre 23291801

Batıhan Babacan 23291565

Date: May 2026

1. INTRODUCTION

On university campuses, thousands of students and staff lose or find personal belongings every day. Traditional methods, such as social media posts or physical boards, are unorganized, unsearchable, and inefficient.

FindIt is a centralized web application designed to help campus communities report lost or found items, search through current listings, and safely get in touch with each other. The objective of this project is to implement a robust database management system that handles relational data smoothly and provides an intuitive platform for campus users.

2. SYSTEM ARCHITECTURE

The system is built using a modern and lightweight monolithic web architecture, making it easy to deploy, run, and demonstrate:

- **Frontend Layer:** Developed using HTML5, CSS3, and JavaScript. The client side communicates with the backend server asynchronously via standard HTTP fetch requests to display data without refreshing the page.
- **Backend Server:** Built using the **Flask** micro-framework in Python. It acts as a RESTful API gateway that handles user authentication, form inputs, database communication, and query formatting.
- **Database Layer:** Powered by a cloud-hosted **PostgreSQL instance on Neon**. PostgreSQL provides full relational database support, transaction safety, and structural constraint enforcement.

3. DATABASE SCHEMA DESIGN

The database layer is directly mapped from our Entity-Relationship (E-R) diagram. The system consists of 6 main tables:

1. **users:** Stores the registration data of campus users. Passwords are encrypted using secure cryptographic hashing function keys (password_hash) before being written to the database.
2. **category:** Defines the item types such as Electronics, Wallets, and Documents to allow better grouping.
3. **item:** Contains the primary specifications of the object, including its name, text description, and associated category identifier.
4. **report:** Manages the contextual metadata of a post. It links an item record to a users record and tracks the report type ('lost' or 'found'), location, date, and current status.
5. **message:** Stores the peer-to-peer chat logs between users. It connects back to the users table via sender_id and receiver_id references.
6. **matches:** Enforces the relationship between a lost report and a found report when a match is generated, tracking the confirmation state via the is_confirmed boolean flag.

4. APPLICATION ENDPOINTS & FUNCTIONALITY

The Flask server exposes multiple backend routes to facilitate application features:

- **Authentication Hooks (/api/register & /api/login):** Handles user onboarding and verifies credentials against the stored database values.
- **Reporting Hook (/api/reports):** A dual-purpose endpoint. A GET request retrieves active campus listings matching the search or type filters. A POST request accepts user payloads to write a new item and its corresponding report to the database inside an integrated transaction block.
- **Messaging Hook (/api/messages):** Allows users to exchange text information regarding an item and retrieves message history filtered by inbox or outbox contexts.
- **Matching Hooks (/api/matches):** Creates match pairs inside the database and handles confirmation updates when users finalize the recovery process.

5. REVENUE AND CONCLUSION

The development of **FindIt** successfully satisfies the core practical requirements of a relational database management application. The database schema enforces data integrity through foreign keys, primary keys, and field constraints while remaining flexible enough to handle the platform's daily operational flow.

The system effectively demonstrates how combining a relational PostgreSQL layer with a clean Flask API enables a reliable utility program that can scale seamlessly for broader campus audiences.